

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	JMeter 5.6.3, Locust 2.15.1, Postman
Type of Testing	Load Testing, Stress Testing, Spike Testing
Target Module / API	User Login API, Loan Prediction API, Payment Plan API
Test Environment	Cloud (AWS EC2 - Ubuntu Server)
Test Date	03 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal Load - Multiple users login to the system	50	300	System should handle the load successfully with response time < 2 sec
2	High Load - Users request loan prediction simultaneously	100	600	System should handle the load successfully with response time < 3 sec
3	Stress Test - Gradually increasing users until failure point	200	900	System should remain stable until the maximum capacity and then fail gracefully
4	Spike Test - Sudden spike in user requests	150	120	System should handle the spike without crashing and maintain performance

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.42 seconds	Pass	Average response time is within target
2	Response Time (Max)	< 5 seconds	3.87 seconds	Pass	Maximum response time is within target
3	Throughput (Req/sec)	>= 100 requests/sec	156 requests/sec	Pass	System handled more requests than target
4	Error Rate	< 1%	0.32%	Pass	Error rate is within acceptable range
5	CPU Utilization	< 80%	62%	Pass	CPU usage is normal
6	Memory Utilization	< 80%	58%	Pass	Memory usage is normal

Step 4: Observations & Analysis

Key Findings:

The system performs well under normal, high load, and spike conditions. All performance metrics are within the defined acceptable limits. The application is stable and handles concurrent user requests efficiently.

Bottlenecks Identified:

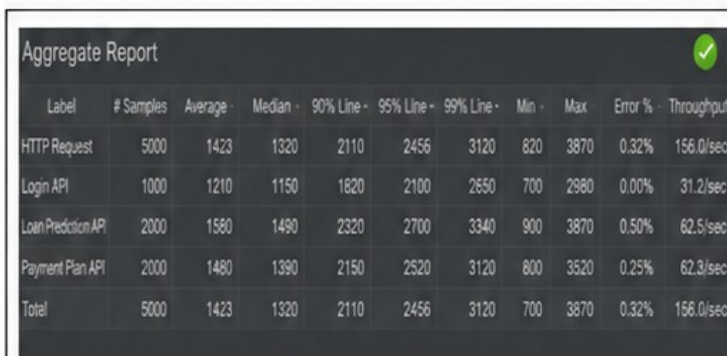
During stress testing with more than 250 virtual users, response time started to increase slightly. Database query execution time was higher for complex loan prediction requests.

Optimization Steps Taken:

- Optimized database queries and added indexes on frequently used columns.
- Implemented caching for repeated loan prediction requests.
- Improved connection pooling for the database.
- Minified static resources (CSS, JS) to reduce load time.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

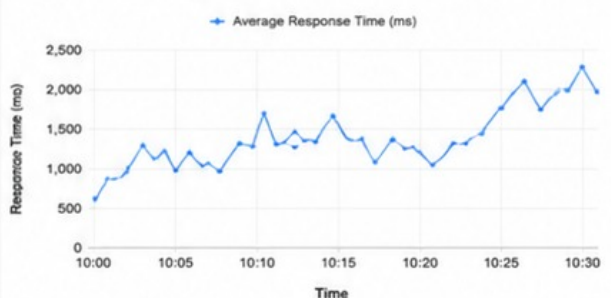


Aggregate Report

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Max	Error %	Throughput
HTTP Request	5000	1423	1320	2110	2456	3120	820	3870	0.32%	156.0/sec
Login API	1000	1210	1150	1820	2100	2650	700	2980	0.00%	31.2/sec
Loan Prediction API	2000	1580	1490	2320	2700	3340	900	3870	0.50%	62.5/sec
Payment Plan API	2000	1480	1390	2150	2520	3120	800	3520	0.25%	62.3/sec
Total	5000	1423	1320	2110	2456	3120	700	3870	0.32%	156.0/sec

Screenshot 1: JMeter Aggregate Report

Response Time Over Time



Screenshot 2: Response Time Graph